Ordenação, Divisão e Conquista

Algoritmos e Estruturas de Dados

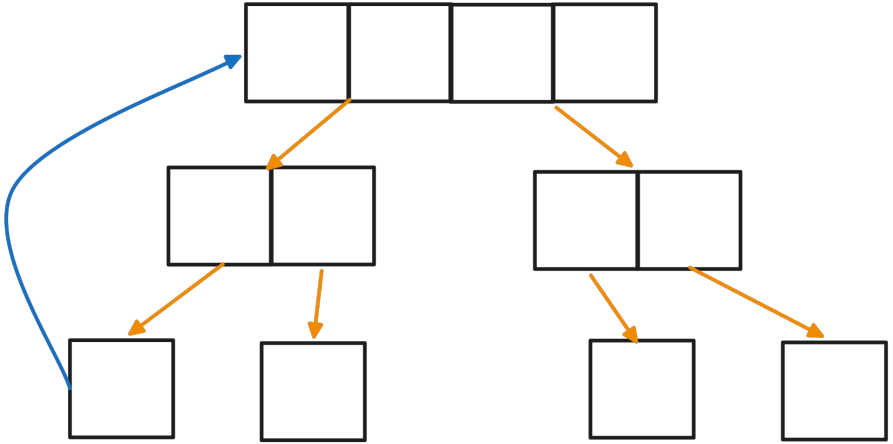
Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Merge Sort

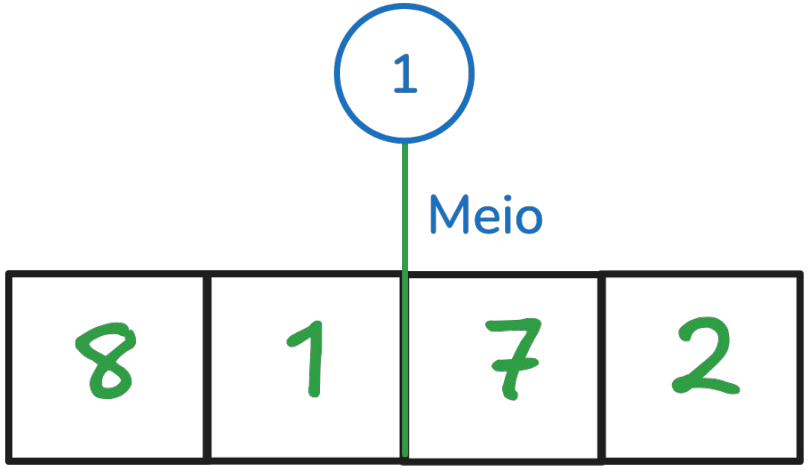
- É um algoritmo de ordenação que segue a premissa do **dividir para conquistar**.
- O MergeSort divide um array em partes iguais até que restem apenas arrays com um único elemento cada, que depois são combinados enquanto comparados até que tenhamos o array base completamente ordenado.



carlos.melo@upe.br

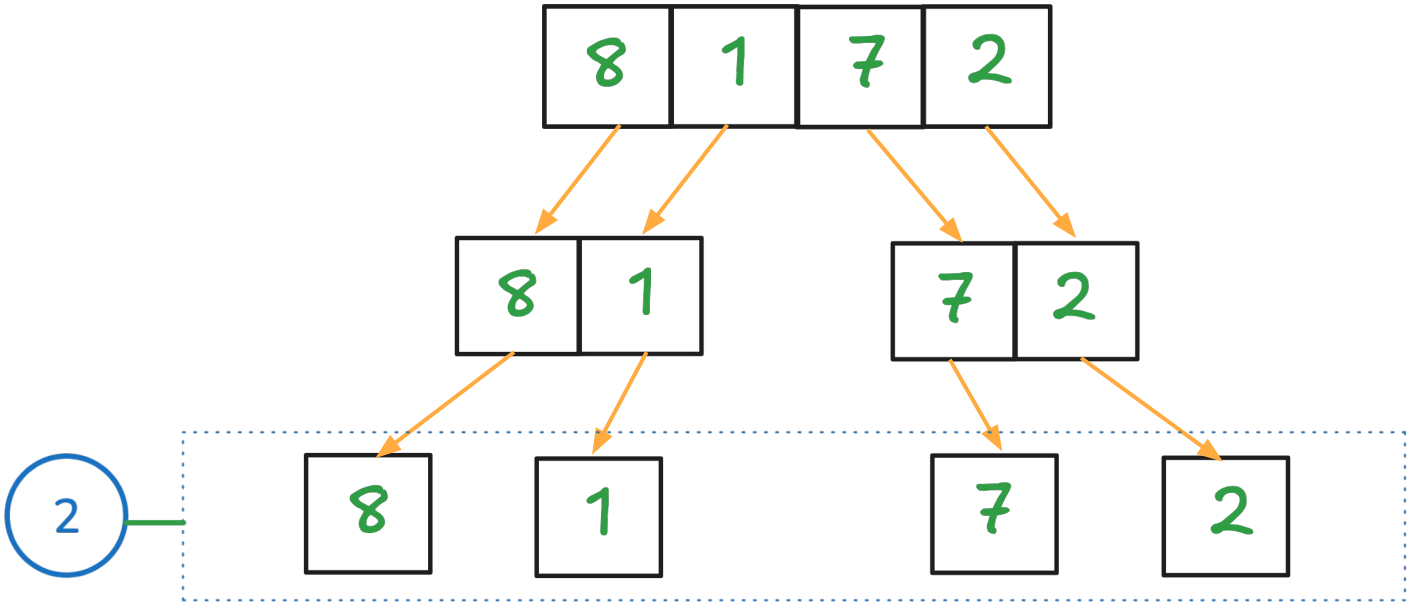
Merge Sort

1. Encontre o índice do meio



Merge Sort (Divisão)

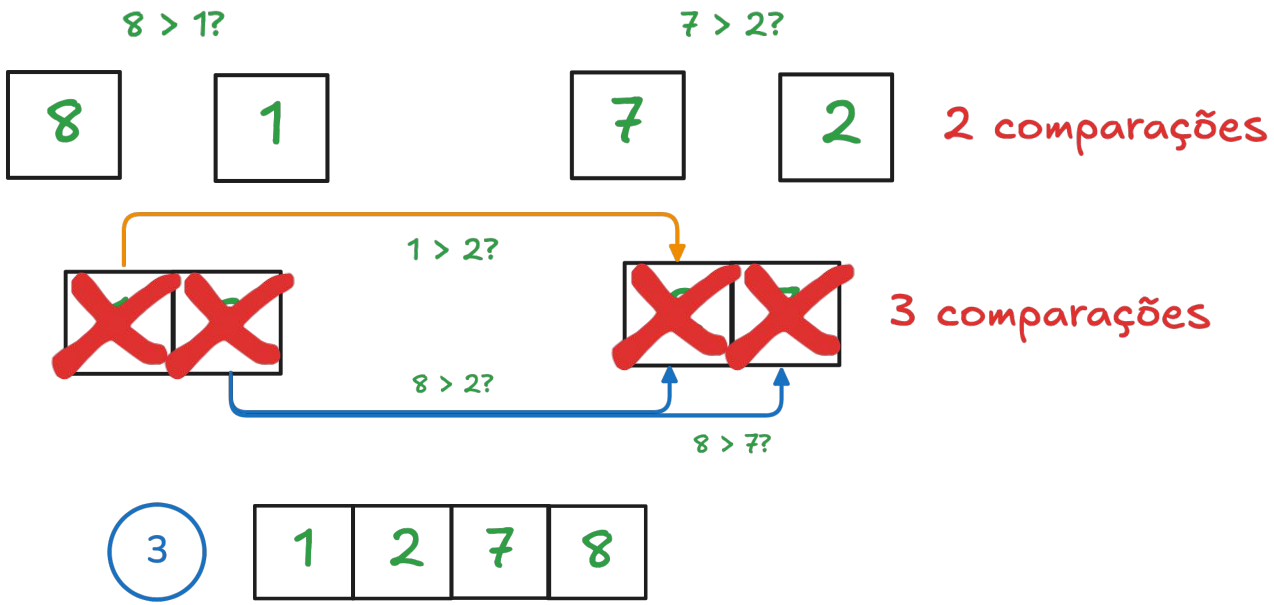
2. Recursivamente divida os arrays a esquerda e a direita do índice do meio até ter em mãos arrays de um único elemento



carlos.melo@upe.br

Merge Sort (Conquista)

3. Continue juntando e trocando elementos até que o array seja reconstruído de forma ordenada



carlos.melo@upe.br

Exercício

- Reproduza o passo a passo do Merge Sort para os arrays à seguir:
  - Quantas comparações foram realizadas em cada um deles?

5	9	6	1	2	11

8	3	1	7	0	10	2

Complexidade do Merge Sort

- O([número de comparações][número de passos])
- No máximo realizamos n comparações
- No máximo realizamos log n passos
- Logo, o MergeSort é **O(n log n)** em operações

Tamanho do Array	1	2	3	4	5	6	7	8	9
Número de Passos	0	1	2	2	3	3	3	3	4
~Número de Comparações	0	1	2	3	4	5	6	7	8

Merge Sort - Código base

```python
def mergesort(items):
    # Caso base, um array com 0 ou 1 elementos que já está ordenado
    if len(items) <= 1:
        return items

    # Caso contrário, encontramos o índice do meio e dividimos o array
    # TODO
    # left =
    # right =

    # Chamamos o mergesort recursivamente com lado esquerdo e direita
    left = mergesort(left)
    right = mergesort(right)

    # Juntamos as duas metades e retornamos
    return merge(left, right)

def merge(left, right):
    # Dado dois arrays ordenados, misturá-los juntos e em ordem
    # retornamos o array ordenado
    # TODO
```

Mergesort - Divisão

- Usamos o operador // do Python para encontrar o ponto do meio. Se o tamanho do array é par, então teremos o índice do meio. Se for ímpar teremos uma metade maior que a outra por um.

```python
list1 = [0, 1, 2, 3]
midpoint1 = len(list1) // 2
print(f'List 1 midpoint: {midpoint1}')

list2 = [4, 5, 6]
midpoint2 = len(list2) // 2
print(f'List 2 midpoint: {midpoint2}')
```

```python
left1 = list1[:midpoint1]
right1 = list1[midpoint1:]
print(f'List 1 left side: {left1}')
print(f'List 1 right side: {right1}')

left2 = list2[:midpoint2]
right2 = list2[midpoint2:]
print(f'List 2 left side: {left2}')
print(f'List 2 right side: {right2}')
```

Merge - Conquista

- A função merge precisa combinar dois arrays ordenados. Podemos fazer isso rastreando um índice em ambas as listas e, quando uma delas se esgotar, adicionando os itens restantes da outra lista.

```python
def merge(left, right):
    merged = []
    left_index = 0
    right_index = 0

    # Nós movemos pelos arrays até nós exaurirmos um
    while left_index < len(left) and right_index < len(right):
        # Se o item a esquerda é maior, adicionamos o da direita
        # e incrementamos o índice
        if left[left_index] > right[right_index]:
            merged.append(right[right_index])
            right_index += 1
        # Caso contrário, adicionamos o item da esquerda e incrementamos
        else:
            merged.append(left[left_index])
            left_index += 1

    # Adicionamos qualquer sobra. Porque dividimos em nosso While,
    # nós sabemos que pelo menos um dele s está vazio:
    # a) estão ordenados
    # b) todos os elementos após o último elemento inserido no array mesclado
    merged += left[left_index:]
    merged += right[right_index:]

    # retornamos o array ordenado e mesclado
    return merged
```

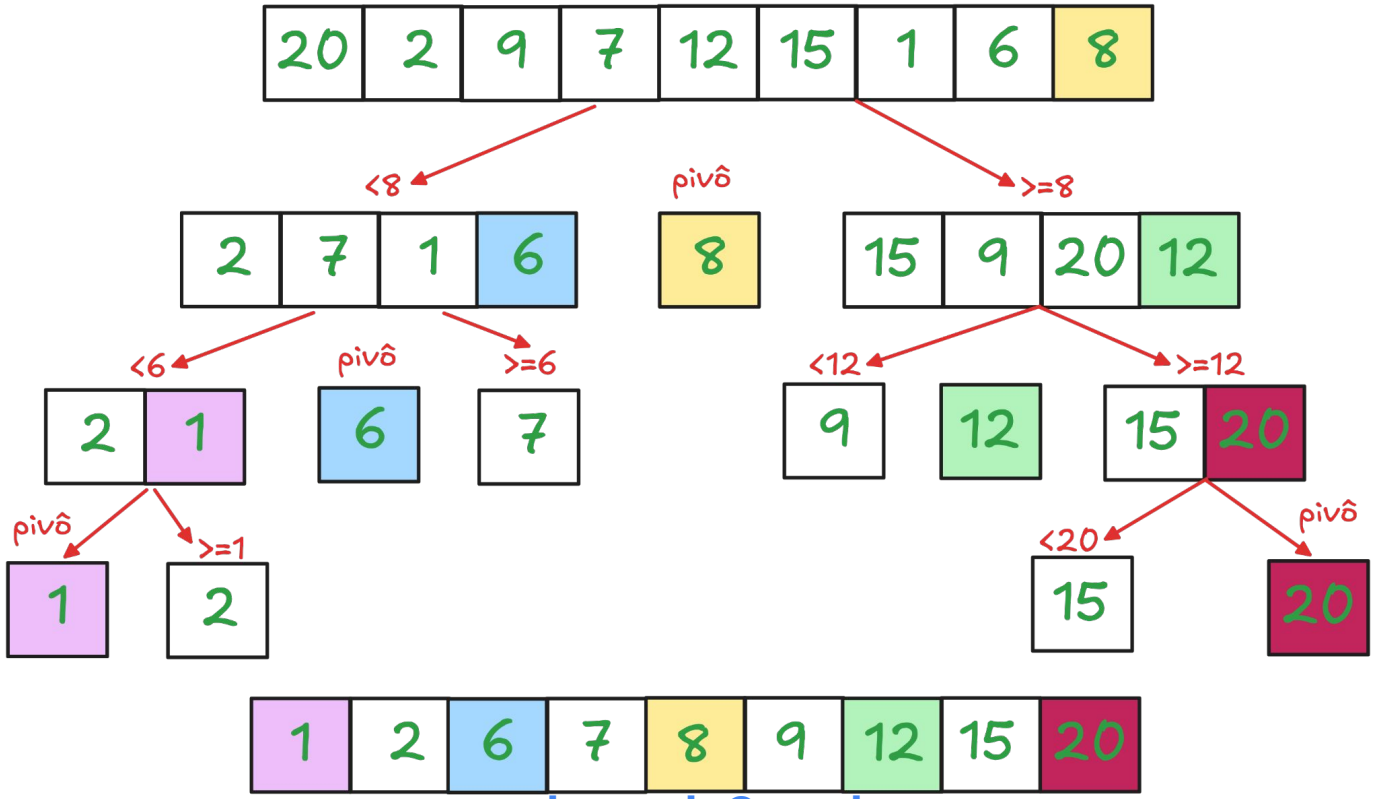
Merge - Conquista

```python
def merge(left, right):
    merged = []
    left_index = 0
    right_index = 0

    # Nós movemos pelos arrays até nós exaurirmos um
    while left_index < len(left) and right_index < len(right):
        # Se o item a esquerda é maior, adicionamos o da direita
        # e incrementamos o índice
        if left[left_index] > right[right_index]:
            merged.append(right[right_index])
            right_index += 1
        # Caso contrário, adicionamos o item da esquerda e incrementamos
        else:
            merged.append(left[left_index])
            left_index += 1

    # Adicionamos qualquer sobra. Porque dividimos em nosso While,
    # nós sabemos que pelo menos um dele s está vazio:
    # a) estão ordenados
    # b) todos os elementos após o último elemento inserido no array mesclado
    merged += left[left_index:]
    merged += right[right_index:]

    # retornamos o array ordenado e mesclado
    return merged
```

Quick Sort

- É um outro algoritmo de ordenação que segue a premissa do **dividir para conquistar**
  - No Merge Sort dividiamos o array ao meio
  - No Quick Sort dividimos usando um pivô
- No Quick Sort escolhemos o pivô e:
  - Elementos menores que o Pivô -> Esquerda
  - Elementos maiores que o Pivô -> Direita
- O pivô fica em sua posição correta definitiva
- Repetimos o processo para os dois lados do pivô (esquerda e direita) até que tudo esteja ordenado

Quick Sort



carlos.melo@upe.br

Quick sort

1. Escolha o pivô
   - [6, 5, 3, 1, 8, 7, 2, **4**]
2. Separe os elementos em relação ao pivô
   - menores à esquerda do pivô
   - maiores à direita do pivô
     i. [3, 1, 2, **4,** 6, 5, 8, 7]
3. O pivô já está na posição correta
4. Repita o processo para cada lado
   - [3, 1, 2] → ordenar
   - [6, 5, 8, 7] → ordenar
5. Fim:
   - [1, 2, 3, 4, 5, 6, 7, 8]

Exercício

- Reproduza o passo a passo do Quick Sort para os arrays à seguir:

1	2	3	4	5	6

7	6	5	4	3	2	1

Implementação do Quick Sort

```python
def quick_sort(numbers, start, end):
    if start < end:
        pivot = partition(numbers, start, end)
        quick_sort(numbers, start, pivot - 1)
        quick_sort(numbers, pivot + 1, end)
```

Implementação do Particionamento

```python
def partition(numbers, start, end):
    pivot = numbers[end]
    delimiter = start - 1

    for index in range(start, end):
        if numbers[index] <= pivot:
            delimiter = delimiter + 1

            temp = numbers[index]
            numbers[index] = numbers[delimiter]
            numbers[delimiter] = temp

    temp = numbers[delimiter + 1]
    numbers[delimiter + 1] = numbers[end]
    numbers[end] = temp

    return delimiter + 1
```

carlos.melo@upe.br

Complexidade do Quick Sort

- Melhor Caso e Caso Médio: O(n log n)
- Pior Caso: O(n^2):
  - Se o array já estiver ordenado em ordem crescente ou decrescente o número de operações cresce substancialmente
- O Merge Sort tem complexidade espacial O(n) e o Quick Sort tem complexidade O(1) pois ordena no lugar sem criar novos arrays/sub arrays.

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS